專案計劃書

機車零件商城與保養管理平台

組別：第五組

組員：梁仲龍、邱恩哲、黃冠穎、謝國瑋

繳交日期：2026 年 11 月 12 日

一、簡介

1.1 背景與動機

機車是台灣最普遍的交通工具，車主在更換零件或保養時，常遇到兩個問題：第一，同一類零件有許多廠牌與規格，車主難以判斷哪些零件適用自己的車型；第二，保養紀錄多半散落在機車行的紙本單據中，車主不容易掌握上次保養的時間與里程，也不清楚更換過哪些零件。

本專案規劃一個結合「零件商城」與「保養管理」的平台。使用者登錄自己的車輛後，系統即可篩選出適用的零件供其購買，並記錄每次保養更換的零件與費用，讓「選車 → 找零件 → 下單 → 保養紀錄」形成完整流程。

1.2 系統目標

- 提供依車型篩選適用零件的查詢功能，降低選購錯誤
- 提供線上購買零件的商城功能，包含購物車、下單與訂單查詢
- 提供車輛保養紀錄管理，記錄保養日期、里程、更換零件與費用
- 採用前端、後端、資料庫三層式架構，後端以 RESTful API 提供服務，未來可延伸支援行動 APP

1.3 目標使用者

使用者	說明
一般會員（customer）	機車車主，可登錄車輛、查詢與購買零件、收藏與評論零件、管理保養紀錄
管理員（admin）	平台經營者，負責零件上架、價格與庫存管理、訂單狀態處理
訪客	未登入的使用者，可瀏覽零件與評論，購買與收藏前需註冊登入

1.4 主要功能

功能模組	功能說明	相關資料表
會員管理	註冊、登入、修改個人資料；密碼以雜湊方式儲存	users
我的車庫	登錄自己的車輛（品牌、車型、車牌），可登錄多台	user_vehicles、brands、motorcycle_models
零件查詢	依分類、製造商、關鍵字查詢；依車庫中的車型篩選適用零件	parts、categories、manufacturers、part_fitments
收藏與評論	收藏零件；對零件評分與評論（每人每零件限一則）	favorites、reviews
商城與訂單	購物車、建立訂單、檢查與扣除庫存、查詢訂單狀態	orders、order_items、parts
保養紀錄	新增每次保養的日期、里程、總金額，以及更換的零件明細	maintenance_records、maintenance_items、service_types
管理員後台	零件上架、修改價格與庫存、更新訂單狀態	parts、orders

1.5 系統範圍與限制

- 本系統為單一商家商城，由平台自行販售零件，不提供多店家進駐
- 保養紀錄僅記錄零件更換項目，每筆保養明細皆對應一個更換的零件
- 金流部分不串接真實付款服務，訂單狀態由管理員手動更新
- 本階段以網頁為主要前端；行動 APP 列為延伸規劃，可共用同一組後端 API

二、專案組織

本組共四名成員，依三層式架構分為 UI（前端）、後端核心、資料庫三個角色，各角色設有主負責人與協助者，並由組員分擔文件與測試工作。

成員	角色	主要職責
黃冠穎（組長）	資料庫、後端核心、專案統籌	資料庫設計與建置、API 設計與開發、系統架構規劃、前後端整合、Bug 修正、專案進度管理
梁仲龍	UI（前端主負責）	介面設計、前端頁面開發、前後端整合、Bug 修正、期末報告與簡報
謝國瑋	UI 協助、系統測試	介面設計與前端頁面開發協助、系統功能測試
邱恩哲	後端協助、測試資料、文件	測試資料建立、後端 API 開發協助、計劃書撰寫與校對、系統功能測試

2.1 三層架構角色負責人

架構層	主負責人	協助
UI（前端）	梁仲龍	謝國瑋
後端核心	黃冠穎	邱恩哲
資料庫	黃冠穎	邱恩哲（測試資料）

三、硬體與軟體需求規格

3.1 開發環境

類別	項目	用途
硬體	個人電腦／筆電：CPU Intel Core i5 同等級以上、記憶體 8 GB 以上、硬碟可用空間 20 GB 以上	程式開發與本機測試
作業系統	Windows 11	開發環境
程式語言	Python 3.11 以上、JavaScript	後端與前端開發
程式編輯器	Visual Studio Code	撰寫前後端程式
資料庫設計	SQL Power Architect	繪製 ERD、產生 DDL
資料庫管理	HeidiSQL 12.22	建立資料表、管理與查詢資料
API 測試	FastAPI 自動產生的互動式文件（Swagger UI，/docs）	測試 API 請求與回應
版本控制	Git、GitHub	組員協作與程式碼管理

3.2 伺服器端

層	技術	說明
後端框架	Python + FastAPI	提供 RESTful API，處理商業邏輯與權限驗證
ASGI 伺服器	Uvicorn	執行 FastAPI 應用程式
資料庫	MariaDB 13.0	關聯式資料庫，使用 InnoDB 引擎以支援外鍵與交易；編碼 utf8mb4
主要套件	PyMySQL（資料庫連線）、passlib／bcrypt（密碼雜湊）、python-jose（JWT）	後端功能所需套件
部署環境	開發階段使用本機；正式上線可部署於雲端主機	—

3.3 使用者端

項目	需求
前端技術	HTML、CSS、JavaScript，搭配 Bootstrap 5 建立響應式介面
瀏覽器	Google Chrome、Microsoft Edge、Safari 等主流瀏覽器之最新版本
裝置	桌上型電腦、筆電、手機皆可透過瀏覽器使用
網路	需連接網際網路
行動 APP（延伸規劃）	未來以 React Native 開發，支援 Android／iOS，共用同一組後端 API

四、資料庫資料表

本系統資料庫名稱為 moto_shop，共 15 張資料表，依功能分為會員與車輛、零件、商城、保養四大類。

#	資料表	中文名稱	說明
1	users	會員	儲存會員帳號資料，以 role 區分一般會員與管理員
2	brands	機車品牌	機車品牌資料（如 KYMCO、SYM、Yamaha）
3	motorcycle_models	車型	各品牌的車型，一個品牌可有多個車型
4	user_vehicles	會員車庫	會員擁有的車輛，一位會員可登錄多台車
5	manufacturers	零件製造商	零件的製造廠商
6	categories	零件分類	零件分類（如 機油、煞車系統、輪胎）
7	parts	零件	商城販售的零件，含售價與庫存
8	part_fitments	零件適用車型	零件與車型的 M:N 中介表，記錄零件適用哪些車型
9	favorites	收藏	會員與零件的 M:N 中介表，記錄收藏的零件
10	reviews	零件評論	會員對零件的評分與評論，每位會員對同一零件限評一次
11	orders	訂單	會員的訂單主檔
12	order_items	訂單明細	訂單與零件的 M:N 中介表，記錄每筆訂單購買的零件與數量
13	maintenance_records	保養紀錄	車輛每一次保養的紀錄
14	maintenance_items	保養明細	一次保養中更換的各項零件
15	service_types	保養項目	保養項目類型（如 更換機油、更換煞車皮）

4.1 設計說明

- M:N 關聯皆拆為中介表（part_fitments、favorites、order_items），並以兩個外鍵組成複合主鍵，避免重複資料
- order_items.unit_price 儲存下單當下的單價，零件日後調價不會影響歷史訂單金額
- users.email 設為 UNIQUE，避免重複註冊；reviews 以 UNIQUE(user_id, part_id) 限制每位會員對同一零件只能評論一次
- 密碼僅儲存雜湊值（user_pwd_hash），不儲存明碼
- 管理員與一般會員共用 users 資料表，以 role 欄位區分權限
- 各表單欄位主鍵皆設為 AUTO_INCREMENT；created_at 預設為 CURRENT_TIMESTAMP，由資料庫自動記錄時間
- 所有資料表使用 InnoDB 引擎，確保外鍵限制生效並支援交易（下單時同時寫入訂單、明細並扣除庫存）

4.2 資料表欄位規格

4.2.1 users（會員）

欄位名稱	資料型別	鍵	空值	預設值	說明
user_id	INT	PK	否	AUTO_INCREMENT	會員編號
user_name	VARCHAR(30)		否		會員名稱
user_pwd_hash	VARCHAR(255)		否		密碼雜湊值（bcrypt），不儲存明碼
email	VARCHAR(50)	UNIQUE	否		登入帳號，不可重複
phone	VARCHAR(15)		否		聯絡電話
role	VARCHAR(20)		否	'customer'	身分：customer／admin
created_at	DATETIME		否	CURRENT_TIMESTAMP	註冊時間

4.2.2 brands（機車品牌）

欄位名稱	資料型別	鍵	空值	預設值	說明
brand_id	INT	PK	否	AUTO_INCREMENT	品牌編號
brand_name	VARCHAR(15)		否		品牌名稱
country	VARCHAR(30)		否		品牌所屬國家

4.2.3 motorcycle_models（車型）

欄位名稱	資料型別	鍵	空值	預設值	說明
model_id	INT	PK	否	AUTO_INCREMENT	車型編號
brand_id	INT	FK → brands	否		所屬品牌
model_name	VARCHAR(30)		否		車型名稱（如 勁戰、JET SL）
released_yr	INT		否		推出年份

4.2.4 user_vehicles（會員車庫）

欄位名稱	資料型別	鍵	空值	預設值	說明
vehicle_id	INT	PK	否	AUTO_INCREMENT	車輛編號
user_id	INT	FK → users	否		車主
model_id	INT	FK → motorcycle_models	否		車型
nickname	VARCHAR(30)		是		車輛暱稱

欄位名稱	資料型別	鍵	空值	預設值	說明
plate_number	VARCHAR(8)		是		車牌號碼
purchase_date	DATE		否		購車日期

4.2.5 manufacturers（零件製造商）

欄位名稱	資料型別	鍵	空值	預設值	說明
manufacturer_id	INT	PK	否	AUTO_INCREMENT	製造商編號
manufacturer_name	VARCHAR(30)		否		製造商名稱
country	VARCHAR(30)		否		製造商所屬國家

4.2.6 categories（零件分類）

欄位名稱	資料型別	鍵	空值	預設值	說明
category_id	INT	PK	否	AUTO_INCREMENT	分類編號
category_name	VARCHAR(30)		否		分類名稱

4.2.7 parts（零件）

欄位名稱	資料型別	鍵	空值	預設值	說明
part_id	INT	PK	否	AUTO_INCREMENT	零件編號
manufacturer_id	INT	FK → manufacturers	否		製造商
category_id	INT	FK → categories	否		分類
part_name	VARCHAR(100)		否		零件名稱
price	INT		否		售價（新台幣）
image_url	VARCHAR(255)		否		零件圖片網址
stock_quantity	INT		否	0	庫存數量

4.2.8 part_fitments（零件適用車型）

欄位名稱	資料型別	鍵	空值	預設值	說明
part_id	INT	PK, FK → parts	否		零件
model_id	INT	PK, FK → motorcycle_models	否		適用車型

4.2.9 favorites（收藏）

欄位名稱	資料型別	鍵	空值	預設值	說明
user_id	INT	PK, FK → users	否		會員

欄位名稱	資料型別	鍵	空值	預設值	說明
part_id	INT	PK, FK → parts	否		收藏的零件
created_at	DATETIME		否	CURRENT_TIMESTAMP	收藏時間

4.2.10 reviews（零件評論）

欄位名稱	資料型別	鍵	空值	預設值	說明
review_id	INT	PK	否	AUTO_INCREMENT	評論編號
user_id	INT	FK → users, UNIQUE(user_id, part_id)	否		評論者
part_id	INT	FK → parts, UNIQUE(user_id, part_id)	否		被評論的零件
rating	DECIMAL(2,1)		否		評分（1.0～5.0）
review_text	VARCHAR(300)		否		評論內容
created_at	DATETIME		否	CURRENT_TIMESTAMP	評論時間

4.2.11 orders（訂單）

欄位名稱	資料型別	鍵	空值	預設值	說明
order_id	INT	PK	否	AUTO_INCREMENT	訂單編號
user_id	INT	FK → users	否		下單會員
order_status	VARCHAR(10)		否	'pending'	狀態：pending／paid／shipped／completed／cancelled
total_amount	INT		否		訂單總金額
created_at	DATETIME		否	CURRENT_TIMESTAMP	下單時間

4.2.12 order_items（訂單明細）

欄位名稱	資料型別	鍵	空值	預設值	說明
order_id	INT	PK, FK → orders	否		所屬訂單
part_id	INT	PK, FK → parts	否		購買的零件
quantity	INT		否		購買數量
unit_price	INT		否		下單當下的單價

4.2.13 maintenance_records（保養紀錄）

欄位名稱	資料型別	鍵	空值	預設值	說明
record_id	INT	PK	否	AUTO_INCREMENT	紀錄編號
vehicle_id	INT	FK → user_vehicles	否		保養的車輛
service_date	DATE		否		保養日期
total_price	INT		否		本次保養總金額
mileage	INT		否		保養時的里程數（公里）

4.2.14 maintenance_items（保養明細）

欄位名稱	資料型別	鍵	空值	預設值	說明
item_id	INT	PK	否	AUTO_INCREMENT	明細編號
record_id	INT	FK → maintenance_records	否		所屬保養紀錄
service_type_id	INT	FK → service_types	否		保養項目
cost	INT		否		此項目費用
part_id	INT	FK → parts	否		更換的零件

4.2.15 service_types（保養項目）

欄位名稱	資料型別	鍵	空值	預設值	說明
service_type_id	INT	PK	否	AUTO_INCREMENT	項目編號
service_name	VARCHAR(30)		否		項目名稱

五、ERD

下圖為使用 SQL Power Architect 繪製之實體關聯圖（Crow's Foot 表示法）。[PK] 為主鍵、[FK] 為外鍵、[PFK] 為同時作為主鍵與外鍵的欄位、[AK] 為唯一鍵；實線表示識別關聯（中介表），虛線表示非識別關聯。

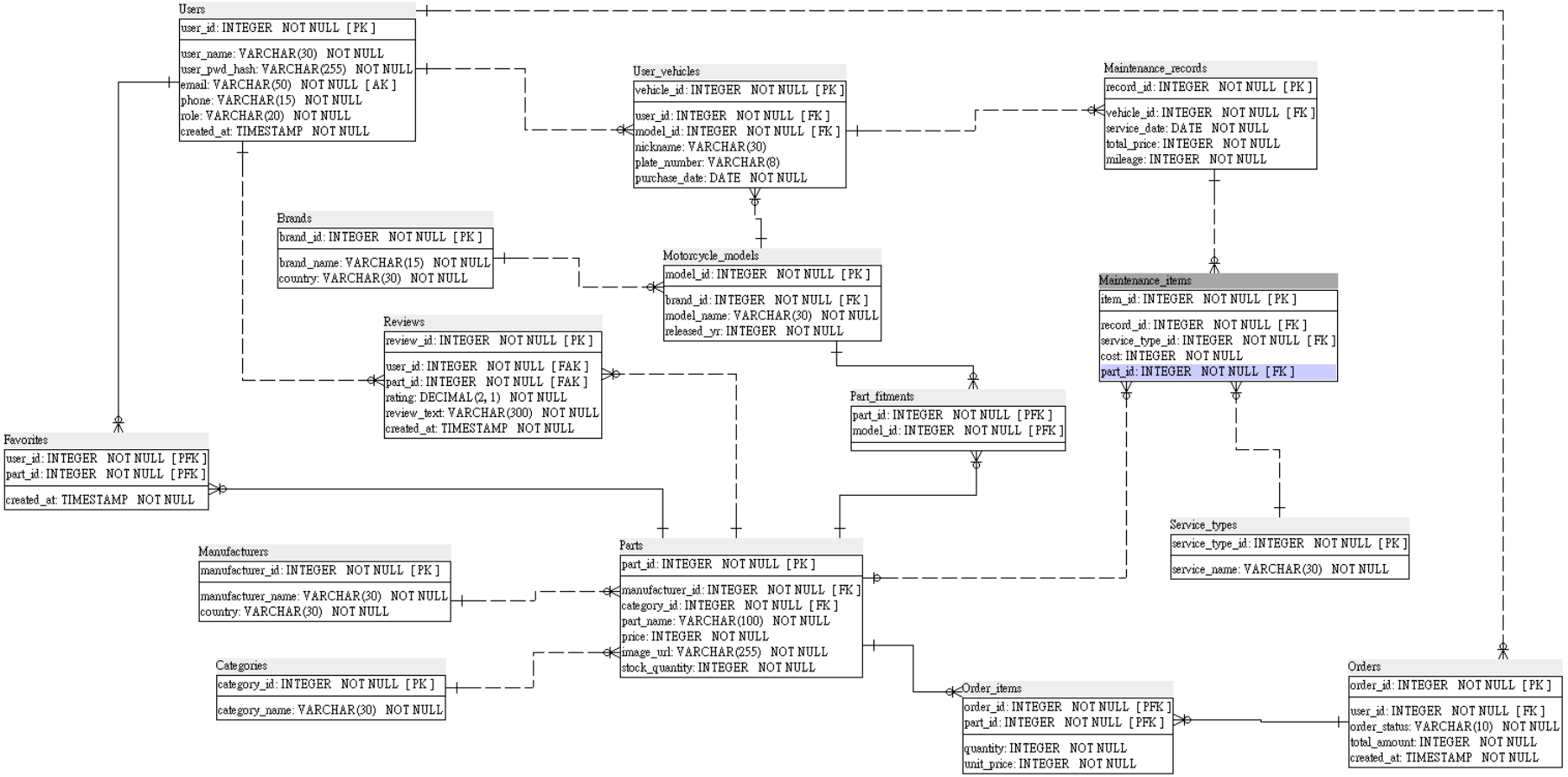


圖 5-1　moto_shop 資料庫 ERD

5.1 關聯說明

父表（一）	子表（多）／中介表	關聯	說明
brands	motorcycle_models	1 : N	一個品牌有多個車型
users	user_vehicles	1 : N	一位會員可登錄多台車
motorcycle_models	user_vehicles	1 : N	一個車型可被多位會員擁有
manufacturers	parts	1 : N	一家製造商生產多個零件
categories	parts	1 : N	一個分類包含多個零件
parts ⇔ motorcycle_models	part_fitments	M : N	零件可適用多個車型，車型也可使用多個零件
users ⇔ parts	favorites	M : N	會員可收藏多個零件，零件可被多人收藏
users	reviews	1 : N	一位會員可撰寫多則評論
parts	reviews	1 : N	一個零件可有多則評論
users	orders	1 : N	一位會員可下多筆訂單
orders ⇔ parts	order_items	M : N	一筆訂單含多個零件，一個零件可出現在多筆訂單
user_vehicles	maintenance_records	1 : N	一台車有多筆保養紀錄
maintenance_records	maintenance_items	1 : N	一次保養包含多個項目
service_types	maintenance_items	1 : N	一種保養項目可出現在多筆明細
parts	maintenance_items	1 : N	一個零件可在多次保養中被更換

六、API 規格書

6.1 共通規範

項目	規範
Base URL	http://localhost:8000/api（開發環境）
資料格式	請求與回應皆使用 JSON，編碼 UTF-8
身分驗證	登入後取得 JWT token，需驗證的 API 須於 Header 帶入：Authorization: Bearer <token>
權限	公開：不需登入；會員：需登入；管理員：需登入且 role 為 admin
API 文件	FastAPI 自動產生互動式文件：http://localhost:8000/docs

HTTP 狀態碼：

狀態碼	意義
200 OK	請求成功
201 Created	資源建立成功
400 Bad Request	參數格式錯誤或不符合規則（如庫存不足）
401 Unauthorized	未登入或 token 無效
403 Forbidden	權限不足（如一般會員呼叫管理員 API）
404 Not Found	資源不存在
409 Conflict	資料衝突（如 email 已註冊、重複評論）
500 Internal Server Error	伺服器內部錯誤

6.2 API 總覽

#	Method	URL	說明	權限
1	GET	/api/health	系統與資料庫連線狀態檢查	公開
2	POST	/api/auth/register	會員註冊	公開
3	POST	/api/auth/login	會員登入，取得 JWT token	公開
4	GET	/api/users/me	取得目前登入會員資料	會員
5	PUT	/api/users/me	修改會員資料	會員
6	GET	/api/brands	取得所有品牌	公開
7	GET	/api/brands/{brand_id}/models	取得某品牌的所有車型	公開
8	GET	/api/categories	取得所有零件分類	公開
9	GET	/api/manufacturers	取得所有製造商	公開
10	GET	/api/parts	查詢零件（可依分類、製造商、車型、關鍵字篩選）	公開
11	GET	/api/parts/{part_id}	取得零件詳細資料（含適用車型）	公開
12	POST	/api/parts	新增零件	管理員

#	Method	URL	說明	權限
13	PUT	/api/parts/{part_id}	修改零件資料、價格、庫存	管理員
14	DELETE	/api/parts/{part_id}	刪除零件	管理員
15	GET	/api/vehicles	取得我的車庫	會員
16	POST	/api/vehicles	新增車輛至車庫	會員
17	DELETE	/api/vehicles/{vehicle_id}	從車庫移除車輛	會員
18	GET	/api/favorites	取得我的收藏	會員
19	POST	/api/favorites	收藏零件	會員
20	DELETE	/api/favorites/{part_id}	取消收藏	會員
21	GET	/api/parts/{part_id}/reviews	取得零件的評論	公開
22	POST	/api/parts/{part_id}/reviews	新增評論（每人每零件限一次）	會員
23	POST	/api/orders	建立訂單（檢查並扣除庫存）	會員
24	GET	/api/orders	取得我的訂單列表	會員
25	GET	/api/orders/{order_id}	取得訂單詳細內容	會員
26	PATCH	/api/orders/{order_id}/status	更新訂單狀態	管理員
27	GET	/api/service-types	取得所有保養項目	公開
28	GET	/api/vehicles/{vehicle_id}/maintenance	取得某車輛的保養紀錄	會員
29	POST	/api/vehicles/{vehicle_id}/maintenance	新增保養紀錄與明細	會員

6.3 主要 API 詳細規格

6.3.1 系統狀態檢查

項目	內容
Method	GET
URL	/api/health
說明	檢查後端服務與資料庫連線是否正常，用於演示 API 往返
權限	公開

Response 範例（成功）：

```
{
  "status": "ok",
  "database": "connected",
  "server_time": "2026-11-12T10:00:00"
}
```

6.3.2 會員登入

項目	內容
Method	POST
URL	/api/auth/login

項目	內容
說明	以 email 與密碼登入，成功後回傳 JWT token
權限	公開

Request Body 範例：

```
{
  "email": "edwin@example.com",
  "password": "mypassword"
}
```

Response 範例（成功）：

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer",
  "user": { "user_id": 1, "user_name": "Edwin", "role": "customer" }
}
```

錯誤回應：

狀態碼	情況
401	email 或密碼錯誤

6.3.3 查詢零件

項目	內容
Method	GET
URL	/api/parts
說明	查詢零件列表，可組合多個篩選條件
權限	公開

請求參數：

參數	位置	型別	必填	說明
category_id	Query	int	否	依分類篩選
manufacturer_id	Query	int	否	依製造商篩選
model_id	Query	int	否	只顯示適用此車型的零件（查詢 part_fitments）
keyword	Query	string	否	零件名稱關鍵字
page	Query	int	否	頁碼，預設 1

Response 範例（成功）：

```
GET /api/parts?model_id=3&category_id=1

{
  "total": 2,
  "page": 1,
  "items": [
```

```
    {
      "part_id": 12,
      "part_name": "全合成機油 10W40 1L",
      "price": 380,
      "stock_quantity": 25,
      "image_url": "https://.../oil.jpg",
      "category": "機油",
      "manufacturer": "Motul"
    }
  ]
}
```

6.3.4 建立訂單

項目	內容
Method	POST
URL	/api/orders
說明	建立訂單。後端以交易（transaction）處理：檢查庫存 → 寫入 orders 與 order_items（記錄當下單價）→ 扣除 parts.stock_quantity；任一步驟失敗則全部回復
權限	會員

Request Body 範例：

```
{
  "items": [
    { "part_id": 12, "quantity": 2 },
    { "part_id": 30, "quantity": 1 }
  ]
}
```

Response 範例（成功）：

```
HTTP 201 Created
{
  "order_id": 105,
  "order_status": "pending",
  "total_amount": 1260,
  "items": [
    { "part_id": 12, "quantity": 2, "unit_price": 380 },
    { "part_id": 30, "quantity": 1, "unit_price": 500 }
  ],
  "created_at": "2026-11-20T14:30:00"
}
```

錯誤回應：

狀態碼	情況
400	庫存不足或數量不合法
401	未登入
404	零件不存在

6.3.5 新增保養紀錄

項目	內容
Method	POST

項目	內容
URL	/api/vehicles/{vehicle_id}/maintenance
說明	為自己的車輛新增一筆保養紀錄與更換零件明細，後端驗證該車輛屬於目前登入會員
權限	會員

Request Body 範例：

```
{
  "service_date": "2026-11-15",
  "mileage": 12500,
  "items": [
    { "service_type_id": 1, "part_id": 12, "cost": 380 },
    { "service_type_id": 3, "part_id": 45, "cost": 650 }
  ]
}
```

Response 範例（成功）：

```
HTTP 201 Created
{
  "record_id": 8,
  "vehicle_id": 2,
  "service_date": "2026-11-15",
  "mileage": 12500,
  "total_price": 1030
}
```

錯誤回應：

狀態碼	情況
403	車輛不屬於目前登入會員
404	車輛、保養項目或零件不存在

七、系統架構圖

本系統採用三層式架構（Three-tier Architecture），前端、後端、資料庫各自獨立，彼此透過明確的介面溝通。

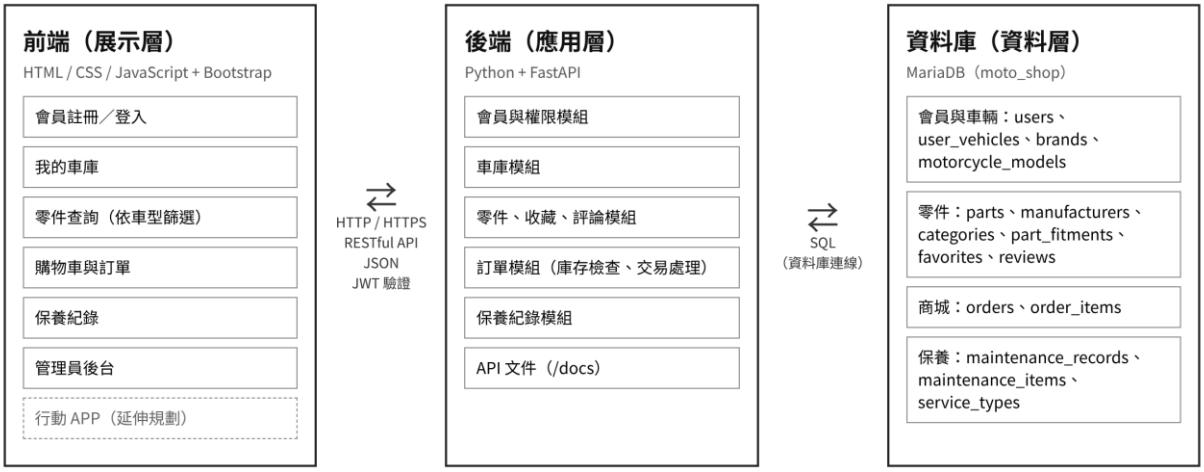


圖 7-1　系統三層式架構

層	技術	職責
展示層（前端）	HTML／CSS／JavaScript＋Bootstrap	提供使用者介面，透過 fetch 呼叫後端 API，並顯示回傳資料
應用層（後端）	Python＋FastAPI	接收 HTTP 請求、驗證身分與權限、執行商業邏輯（如庫存檢查與扣除）、存取資料庫並回傳 JSON
資料層（資料庫）	MariaDB	儲存系統資料，以主鍵、外鍵、唯一限制確保資料完整性

7.1 API 往返流程（以查詢零件為例）

- 使用者在前端選擇車型，瀏覽器送出 GET /api/parts?model_id=3
- FastAPI 接收請求，驗證參數後組成 SQL，查詢 parts 與 part_fitments
- MariaDB 回傳查詢結果，後端整理成 JSON 格式回應
- 前端收到 JSON 後，將零件列表顯示在頁面上

前端不直接連線資料庫，所有資料存取皆經由後端 API，因此網頁與未來的行動 APP 可以共用同一組 API。

八、專案時程：甘特圖

專案期間為 2026/11/2 至 2026/12/22，以 GanttProject 繪製。重要里程碑（◆）：11/12 繳交計劃書、12/4 核心功能完成、12/22 期末 Demo。

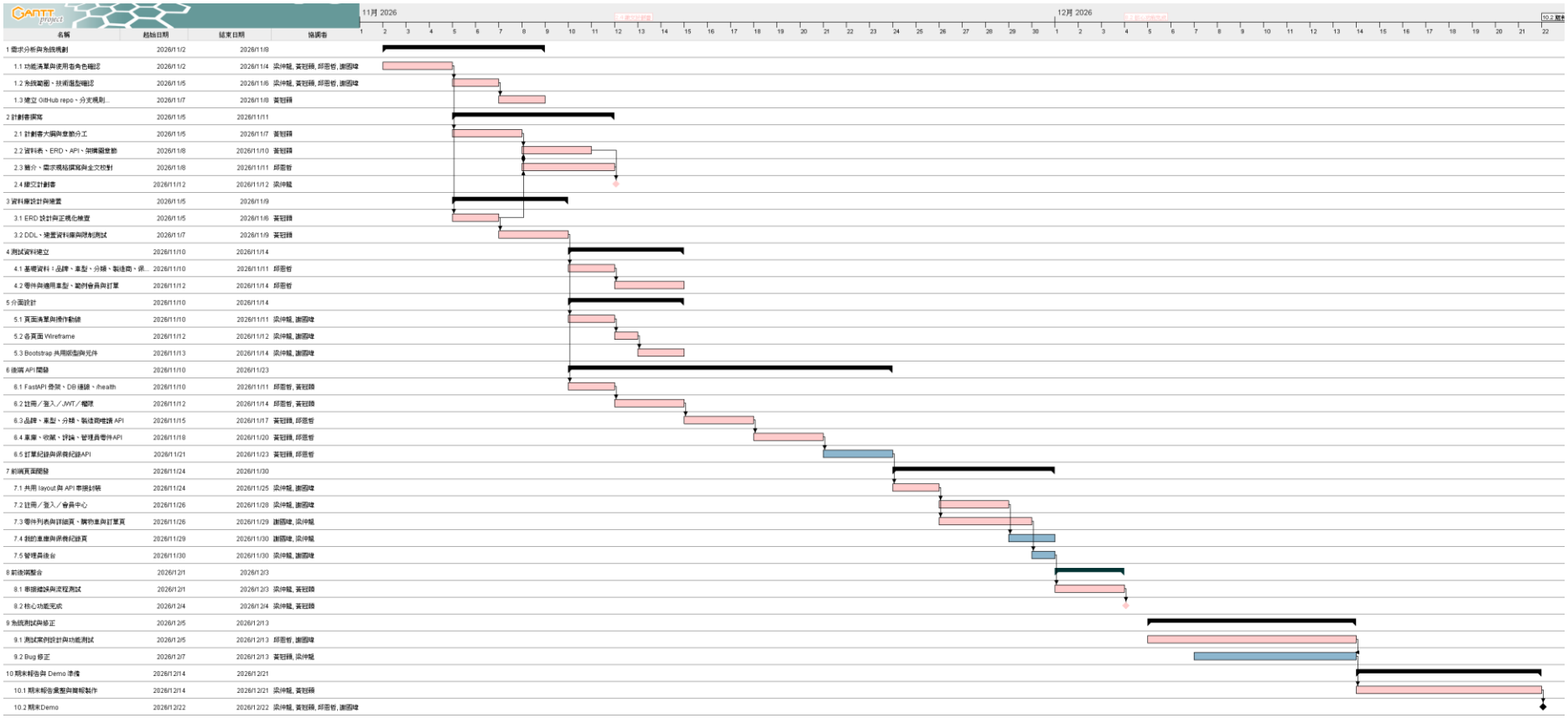


圖 8-1　專案甘特圖（2026/11/2～2026/12/22）

8.1 任務明細

#	任務	開始	結束	負責人
1	需求分析與系統規劃	11/2	11/8	全員
1.1	功能清單與使用者角色確認	11/2	11/4	全員
1.2	系統範圍、技術選型確認	11/5	11/6	全員
1.3	建立 GitHub repo、分支規則	11/7	11/8	黃冠穎
2	計劃書撰寫	11/5	11/12	黃冠穎、邱恩哲、梁仲龍
2.1	計劃書大綱與章節分工	11/5	11/7	黃冠穎
2.2	資料表、ERD、API、架構圖章節	11/8	11/10	黃冠穎
2.3	簡介、需求規格撰寫與全文校對	11/8	11/11	邱恩哲
2.4	◆ 繳交計劃書	11/12	11/12	梁仲龍
3	資料庫設計與建置	11/5	11/9	黃冠穎
3.1	ERD 設計與正規化檢查	11/5	11/6	黃冠穎
3.2	DDL、建置資料庫與限制測試	11/7	11/9	黃冠穎
4	測試資料建立	11/10	11/14	邱恩哲
4.1	基礎資料：品牌、車型、分類、製造商、保養項目	11/10	11/11	邱恩哲
4.2	零件與適用車型、範例會員與訂單	11/12	11/14	邱恩哲
5	介面設計	11/10	11/14	梁仲龍、謝國瑋
5.1	頁面清單與操作動線	11/10	11/11	梁仲龍、謝國瑋
5.2	各頁面 Wireframe	11/12	11/12	梁仲龍、謝國瑋
5.3	Bootstrap 共用版型與元件	11/13	11/14	梁仲龍、謝國瑋
6	後端 API 開發	11/10	11/23	黃冠穎、邱恩哲
6.1	FastAPI 骨架、DB 連線、/health	11/10	11/11	黃冠穎、邱恩哲
6.2	註冊／登入／JWT／權限	11/12	11/14	黃冠穎、邱恩哲

#	任務	開始	結束	負責人
6.3	品牌、車型、分類、製造商唯讀 API	11/15	11/17	黃冠穎、邱恩哲
6.4	車庫、收藏、評論、管理員零件 API	11/18	11/20	黃冠穎、邱恩哲
6.5	訂單紀錄與保養紀錄 API	11/21	11/23	黃冠穎、邱恩哲
7	前端頁面開發	11/24	11/30	梁仲龍、謝國瑋
7.1	共用 layout 與 API 串接封裝	11/24	11/25	梁仲龍、謝國瑋
7.2	註冊／登入／會員中心	11/26	11/28	梁仲龍、謝國瑋
7.3	零件列表與詳細頁、購物車與訂單頁	11/26	11/29	謝國瑋、梁仲龍
7.4	我的車庫與保養紀錄頁	11/29	11/30	謝國瑋、梁仲龍
7.5	管理員後台	11/30	11/30	梁仲龍、謝國瑋
8	前後端整合	12/1	12/4	梁仲龍、黃冠穎
8.1	串接除錯與流程測試	12/1	12/3	梁仲龍、黃冠穎
8.2	◆ 核心功能完成	12/4	12/4	梁仲龍、黃冠穎
9	系統測試與修正	12/5	12/13	全員
9.1	測試案例設計與功能測試	12/5	12/13	邱恩哲、謝國瑋
9.2	Bug 修正	12/7	12/13	黃冠穎、梁仲龍
10	期末報告與 Demo 準備	12/14	12/22	全員
10.1	期末報告彙整與簡報製作	12/14	12/21	梁仲龍、黃冠穎
10.2	◆ 期末 Demo	12/22	12/22	全員

九、工作細分

依老師要求，以下列出 UI、後端核心、資料庫三個角色的負責同學與具體工作項目，並與甘特圖任務編號對應。

9.1 角色負責人

角色	負責同學	對應甘特圖任務
UI（前端）	梁仲龍（主負責）、謝國瑋（協助）	5 介面設計、7 前端頁面開發、8 前後端整合
後端核心	黃冠穎（主負責）、邱恩哲（協助）	6 後端 API 開發、8 前後端整合
資料庫	黃冠穎（主負責）、邱恩哲（測試資料）	3 資料庫設計與建置、4 測試資料建立

9.2 各成員工作項目

成員	工作項目
黃冠穎（組長）	• 1.3 建立 GitHub repo • 2.1～2.2 計劃書大綱、資料表／ERD／API／架構圖章節 • 3.1～3.2 資料庫設計與建置 • 6.1～6.5 後端 API 開發 • 8.1 前後端整合、9.2 Bug 修正、10.1 期末報告與簡報
梁仲龍	• 5.1～5.3 介面設計 • 7.1～7.5 前端頁面開發 • 8.1 前後端整合、9.2 Bug 修正 • 2.4 繳交計劃書、10.1 期末報告與簡報
謝國瑋	• 5.1～5.3 介面設計 • 7.1～7.5 前端頁面開發 • 9.1 測試案例設計與功能測試
邱恩哲	• 2.3 簡介、需求規格撰寫與全文校對 • 4.1～4.2 測試資料建立 • 6.1～6.5 後端 API 開發協助 • 9.1 測試案例設計與功能測試
全員	• 1.1～1.2 需求分析與系統規劃、10.2 期末 Demo